

CREDIT CARD FRAUD DETECTION

Submitted By: Noman Sarwar

Internship Domain: Cyber Security

Email Address: nomansarwarchadhar11@gmail.com

[Google Collab Code Link](#)

1. Introduction

Credit card fraud has become one of the major concerns in the financial industry. Fraudulent transactions can result in significant financial losses for both customers and financial institutions. Machine learning techniques provide an efficient way to detect suspicious transactions automatically.

The purpose of this project is to build a machine learning model capable of identifying fraudulent credit card transactions based on transaction amount, time, and anonymized features. The model helps distinguish between genuine and fraudulent transactions and contributes to improving financial security.

2. Objectives

The main objectives of this project are:

- To analyze credit card transaction data.
- To preprocess and clean the dataset.
- To handle the class imbalance problem.
- To train machine learning models for fraud detection.
- To evaluate model performance using suitable metrics.
- To identify fraudulent transactions accurately.

3. Dataset Description

The project uses the Credit Card Fraud Detection dataset.

Features Included

- Time
- Amount
- V1 to V28 (anonymized variables)
- Class

Target Variable

- 0 = Normal Transaction
- 1 = Fraudulent Transaction

The dataset contains highly imbalanced classes where fraudulent transactions represent only a small portion of total transactions.

4. Tools and Technologies Used

The following tools and libraries were used:

- Python
- Google Colab
- Pandas
- NumPy

- Matplotlib
- Seaborn
- Scikit-Learn
- Imbalanced-Learn (SMOTE)
- XGBoost

5. Methodology

The project was completed through the following steps:

1. Loading the dataset.
2. Exploring transaction data.
3. Preprocessing and scaling features.
4. Splitting data into training and testing sets.
5. Handling class imbalance using SMOTE.
6. Training machine learning models.
7. Evaluating model performance.
8. Predicting fraudulent transactions.

6. Data Preprocessing

Before training the model, the dataset was examined for missing values and feature scaling was performed.

StandardScaler was used to normalize transaction amounts and improve model performance.

Data preprocessing ensures that the machine learning algorithms receive properly formatted data.

7. Handling Class Imbalance

One of the biggest challenges in fraud detection is the imbalance between normal and fraudulent transactions.

To overcome this problem, Synthetic Minority Oversampling Technique (SMOTE) was applied.

SMOTE generates synthetic samples for the minority class and balances the dataset, allowing the model to learn fraud patterns more effectively.

8. Model Training

Three machine learning algorithms were considered:

Logistic Regression

Logistic Regression provides a simple and interpretable baseline model.

Random Forest

Random Forest combines multiple decision trees and improves prediction accuracy.

XGBoost

XGBoost is an advanced boosting algorithm that often produces excellent results for classification problems.

Among these models, Random Forest and XGBoost generally provide superior performance in detecting fraud.

9. Model Evaluation

The following evaluation metrics were used:

Accuracy

Measures the overall prediction performance.

Precision

Indicates how many predicted fraud transactions were actually fraudulent.

Recall

Measures the ability to identify fraudulent transactions correctly.

F1-Score

Provides a balance between precision and recall.

ROC-AUC Score

Evaluates the model's capability to distinguish between fraud and legitimate transactions.

10. Results and Discussion

After applying SMOTE and training the models, the fraud detection system achieved strong performance.

Random Forest and XGBoost showed better classification capabilities compared with Logistic Regression.

The model successfully identified most fraudulent transactions while minimizing false positives.

These results demonstrate that machine learning techniques can effectively support fraud prevention systems used in financial institutions.

11. Conclusion

This project demonstrated the use of machine learning techniques for detecting fraudulent credit card transactions.

The data was preprocessed and class imbalance was handled using SMOTE. Multiple classification models were trained and evaluated using different performance metrics.

The results indicate that Random Forest and XGBoost are effective approaches for identifying fraud and improving transaction security.

12. Future Improvements

The system can be enhanced by:

- Using deep learning techniques.
- Applying real-time fraud detection systems.
- Incorporating additional transaction features.
- Using larger datasets.
- Implementing ensemble methods for improved accuracy.

14. Python Code

Import Libraries

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns


from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import classification_report

from sklearn.metrics import confusion_matrix
```

Load Dataset

```
data = pd.read_csv("creditcard.csv")


print(data.head())

print(data.shape)
```

Feature Scaling

```
scaler = StandardScaler()


data['scaled_amount'] = scaler.fit_transform(
data['Amount'].values.reshape(-1,1)
)
```

```
data.drop(['Amount'], axis=1, inplace=True)
```

Split Dataset

```
X = data.drop('Class', axis=1)
```

```
y = data['Class']
```

```
X_train, X_test, y_train, y_test = train_test_split(  
X,  
y,  
test_size=0.2,  
random_state=42,  
stratify=y  
)
```

Apply SMOTE

```
from imblearn.over_sampling import SMOTE
```

```
smote = SMOTE(random_state=42)
```

```
X_train_resampled, y_train_resampled = smote.fit_resample(  
X_train,  
y_train  
)
```

Random Forest Model


```
from sklearn.ensemble import RandomForestClassifier
```

```
rf_model = RandomForestClassifier(  
n_estimators=100,  
random_state=42  
)
```

```
rf_model.fit(X_train_resampled, y_train_resampled)
```

Prediction

```
y_pred = rf_model.predict(X_test)
```

Classification Report

```
print(classification_report(y_test, y_pred))
```

Confusion Matrix

```
cm = confusion_matrix(y_test, y_pred)
```

```
print(cm)
```

Save Model

```
import joblib
```

```
joblib.dump(rf_model, "fraud_detection_model.pkl")
```

Final Remarks

This project highlights the practical application of machine learning techniques in cybersecurity and financial fraud detection. By leveraging data preprocessing, class balancing, and advanced classification algorithms, an effective fraud detection system can be developed to enhance transaction security and minimize financial losses.

[Code Link](#)